

I want to compile and export smolvlm500m model using executorch for qcs 6490 model

Charted ExecuTorch compilation pipeline while acknowledging knowledge limitations >

Given the history we have here, let me give you a solid pipeline walkthrough — but I'll flag script paths as "verify in repo" rather than assert them, since that's bitten us before.

SmolVLM-500M → ExecuTorch → QCS6490

Prerequisites

```
bash

# Clone ExecuTorch
git clone https://github.com/pytorch/executorch.git
cd executorch
git submodule update --init --recursive

# Install with QNN backend support
./install_requirements.sh --pybind qnn

# Install QNN SDK (Qualcomm AI Engine Direct)
# Set env vars:
export QNN_SDK_ROOT=/path/to/qairt/<version>
export PYTHONPATH=$PYTHONPATH:$(pwd)
```

Step 1 — Export the Model

SmolVLM is a VLM with separate vision encoder + language model components. You need to export them **separately** since they run on different backends (NPU vs CPU).

```
python
```

```

import torch
from transformers import AutoProcessor, AutoModelForVision2Seq

model_id = "HuggingFaceTB/SmolVLM-500M-Instruct"
processor = AutoProcessor.from_pretrained(model_id)
model = AutoModelForVision2Seq.from_pretrained(model_id, torch_dtype=torch.float32)
model.eval()

# --- Export vision encoder ---
image_encoder = model.model.vision_model # verify attr path for SmolVLM
pixel_values = torch.randn(1, 3, 384, 384) # adjust to model's expected input

ep_vision = torch.export.export(
    image_encoder,
    args=(pixel_values,),
)

# --- Export language model (decoder) ---
# This needs careful handling of KV cache and dynamic shapes

```

⚠ **Check the actual model attribute names for SmolVLM-500M —**

`model.model.vision_model` is a guess. Inspect `model.named_modules()` or the HF config.

Step 2 — Quantization (torchao)

For QNN HTP (NPU), the recommended config is **16a8w** (most stable) or **8a8w**:

```

python

from torchao.quantization.quant_api import (
    quantize_,
    Int8WeightOnlyQuantizer, # for 8w
)
from executorch.backends.qualcomm.quantizer.quantizer import QnnQuantizer
from executorch.backends.qualcomm.quantizer.custom_annotation import (
    custom_annotate_llama_last_conv_head_scale
)
from torch.ao.quantization.quantize_pt2e import (
    prepare_pt2e, convert_pt2e
)

```

```

quantizer = QnnQuantizer()
# For 16a8w (recommended starting point):
quantizer.set_global_op_quant_config(
    # use per-channel weight, per-tensor activation
)

prepared = prepare_pt2e(exported_program, quantizer)
# run calibration data through `prepared`
converted = convert_pt2e(prepared)

```

From your past work: **8a8w and 16a16w produced garbage outputs on the HTP emulator** — start with **16a8w** per-channel weight config first.

Step 3 — QNN Partitioning & Lowering

```

python

from executorch.backends.qualcomm.partition.qnn_partitioner import QnnPartitioner
from executorch.backends.qualcomm.utils.utils import (
    generate_htp_compiler_spec,
    generate_qnn_executorch_compiler_spec,
)
from executorch.exir import to_edge, EdgeCompileConfig

# Build compiler spec targeting QCS6490 (SM7325 → HTP v69)
backend_options = generate_htp_compiler_spec(
    is_fp16=False, # set True if using fp16 path
)
compiler_specs = generate_qnn_executorch_compiler_spec(
    soc_model=QcomChipset.SM7325, # QCS6490
    backend_options=backend_options,
    debug=False,
    saver=False,
)

partitioner = QnnPartitioner(compiler_specs)

# Lower to edge IR
edge_program = to_edge(
    converted,

```

```

        compile_config=EdgeCompileConfig(_check_ir_validity=False),
    )

# Delegate to QNN
edge_program = edge_program.to_backend(partitioner)

# Export to .pte
exec_program = edge_program.to_executorch()

with open("smolvlm_500m.pte", "wb") as f:
    exec_program.write_to_file(f)

```

Step 4 — Heterogeneous Strategy (CPU prefill / NPU decode)

Based on your earlier work on this:

```

python

# Prefill graph → CPU (avoids NPU input shape constraints)
# Decode graph → NPU (benefits most from NPU throughput)

# Export two separate .pte files:
# smolvlm_prefill_cpu.pte → no QnnPartitioner, just EdgeIR
# smolvlm_decode_npu.pte → with QnnPartitioner

```

This requires splitting the attention forward pass into prefill vs. decode paths with static KV cache shapes for the NPU graph.

Step 5 — Validate on HTP Emulator (x86)

```

bash

# Build the QNN emulator runner
cd executorch
cmake -DEXECUTORCH_BUILD_QNN=ON \
      -DQNN_SDK_ROOT=$QNN_SDK_ROOT \
      -DEXECUTORCH_BUILD_EXTENSION_RUNNER_UTIL=ON \
      -Bbuild_qnn .
cmake --build build_qnn -j$(nproc)

# Run on emulator

```

```
./build_qnn/backends/qualcomm/qnn_executor_runner \  
  --model_path smolvlm_500m.ptc \  
  --input_list inputs.txt
```

Step 6 — On-Device Push (QCS6490)

```
bash  
  
adb push smolvlm_500m.ptc /data/local/tmp/  
adb push libQnnHtp.so /data/local/tmp/ # from QNN SDK  
  
adb shell "cd /data/local/tmp && \  
  LD_LIBRARY_PATH=. ./qnn_executor_runner \  
  --model_path smolvlm_500m.ptc"
```

Key Pitfalls to Watch

Issue	Notes
Script paths	Verify against your exact ExecuTorch commit — don't trust assumed paths
<code>SM7325</code> chipset enum	Verify this is the correct <code>QcomChipset</code> enum value for QCS6490 in your SDK version
Vision encoder input shape	SmolVLM uses tiled images; input may be <code>(N_tiles, 3, H, W)</code> not <code>(1, 3, H, W)</code>
Dynamic shapes	<code>torch.export</code> needs explicit <code>dynamic_shapes</code> for variable sequence lengths
8a8w garbage output	Stick to 16a8w until calibration/annotation issues are resolved

Want me to go deeper on any specific step — e.g., the dynamic shapes export, splitting prefill/decode graphs, or the calibration loop for quantization?

<https://github.com/pytorch/executorch>

We will come to cpu prefill later , first explain normal plrting with what files we run for compilation what code fllw will be and how it works

Investigated repository structure to verify SmolVLM porting workflow accurately >

Good call giving me the repo — let me verify the actual structure rather than guess at paths.

Pivoted to GitHub API for repository exploration >

Let me use the GitHub API to explore the actual structure.

Investigating llama and qwen2_5 decoder architectures systematically >

No SmolVLM script in `oss_scripts`. Let me check the broader repo and the LLM directory structure.

Cataloging llama directory contents, troubleshooting qwen2_5 symlink >

Search API is rate-limited unauthenticated. Let me read the actual canonical scripts — the LLM directory and the README that documents the flow.

Reconciling outdated information with newly supported SmolVLM integration >

The `llama` directory is the canonical reference (and `qwen2_5` just reuses it). It even has a `qnn_multimodal_runner.cpp`. Let me read the main script and its README to trace the real compile flow.

Verified SmolVLM support and traced unified compilation entry point >

SmolVLM-500M **is** supported now — and crucially, there's no separate script. Everything routes through one entry point: `llama.py`. Let me read the actual code to trace the compile flow.

Deciphered modular compilation architecture with separate component files >

Now the key orchestrator — `export_llama()` — and where `MultiModalManager` lives.

Tracing SmolVLM registration and vision encoder sourcing >

Let me confirm the SmolVLM config registration and the model component files.

Synthesized SmolVLM compilation pipeline and verified code flow architecture >

I dug through the actual repo, and there's an important update to what I told you earlier: **SmolVLM-500M is now fully supported**, and there's **no separate script to write**. My earlier suggestion to hand-roll `torch.export` for the vision encoder and LM was wrong for the current main branch — the whole thing routes through one harness. Here's how the real flow works.

One entry point, not a custom script

Everything goes through `examples/qualcomm/oss_scripts/llama/llama.py`. SmolVLM is selected purely by a flag:

```
bash

python examples/qualcomm/oss_scripts/llama/llama.py \
  -b build-android -s ${SERIAL_NUM} -m ${SOC_MODEL} \
  --decoder_model smolvlm_500m_instruct \
  --model_mode hybrid --prefill_ar_len 16 --max_seq_len 1024 \
  --prompt "Can you describe this image?" \
  --image_path "https://.../some.jpg"
```

The model is registered declaratively in `llama/__init__.py`:

```
python

@register_llm_model("smolvlm_500m_instruct", modality_encoders=SmolVLMEncoder)
@dataclass(init=False, frozen=True)
class SmolVLM_500M(LLMModelConfig):
    repo_id = "HuggingFaceTB/SmolVLM-500M-Instruct"
    params_path = ".../models/smolvlm/500M_config.json"
    convert_weights = convert_smolvlm_weights
    quant_recipe = SmolVLMQuantRecipe
    num_sharding = 1
    ...
```

So the config (weights, quant recipe, vision encoder class) is all looked up from `SUPPORTED_LLM_MODELS[args.decoder_model]`. You don't touch export code.

Code flow (top to bottom)

`main()` → `export_llama(args)` → `compile(...)` → `inference(...)`

`export_llama()` (the orchestrator) does:

1. Validates args, looks up `decoder_model_config = SUPPORTED_LLM_MODELS["smolvlm_500m_instruct"]`.
2. Picks the decoder pte base name from `--model_mode: kv` → `kv_llama_qnn`, `hybrid` → `hybrid_llama_qnn`.
3. Builds a **per-modality** `pte_filenames` dict — this is the key VLM detail. A VLM is split into separate graphs, each becoming its own `.pte`:
 - `vision_encoder_qnn.pte` (SigLIP/Idefics3 encoder + projector)
 - `tok_embedding_qnn.pte` (token embedding table)
 - `hybrid_llama_qnn.pte` (the text decoder — reuses `static_llama`)
4. Builds the tokenizer (`TokenizerWrapper`) and calibration data (`DatasetBuilder`, from your prompt or `--tasks`).
5. Sets `is_multimodal = hasattr(config, VISION_ENCODER)` → True for SmolVLM.
6. Calls `compile(...)`, then `inference(...)` unless `--compile_only`.

`compile()` is where the AOT lowering happens, driven by a `MultiModalManager`:

1. For each modality (`VISION_ENCODER`, `TOK_EMBEDDING`, `TEXT_DECODER`) it builds a QNN compile spec via `generate_hip_compiler_spec(...)` + `generate_qnn_executortorch_compiler_spec(soc_model=get_soc_to_chipset_map()[args.soc_model], ...)`.
2. `multi_modal_mgr.quantize(calibration_data, ...)` runs PTQ per component using the model's `quant_recipe`.
3. `multi_modal_mgr.compile(compile_specs, pte_filenames, ...)` runs the actual `torch.export` → `quantize` → `to_edge` → `QnnPartitioner` → `to_backend` → `to_executortorch` per component and writes the `.pte` files into `--artifact`.

Note the per-modality spec differences the code bakes in: the **vision encoder defaults to fp16** unless single-image quantization applies (encoders are quant-sensitive), while the **decoder and tok-embedding use weight sharing** (`use_weight_sharing=not enable_x86_64`).

What you actually get out

Multiple `.pte` files in `--artifact`, plus the C++ runner `qnn_multimodal_runner.cpp` (VLM path, vs `qnn_llama_runner.cpp` for text-only) that loads all three and does the encode → embedding-fusion → autoregressive-decode pipeline on device.

To just produce artifacts without a device, add `--compile_only`. To validate on x86 host, add `--enable_x86_64` (which disables shared buffer and weight sharing, as the spec code shows).

Two things to verify for QCS6490 specifically

1. **The `-m` SOC value:** pass whatever key maps to QCS6490 in `get_soc_to_chipset_map()` — check that dict rather than trusting a guessed enum (I won't assert `SM7325` again).
2. **HTP version gating:** the README lists minimums — weight sharing between prefill/decode and 16-bit matmul both need **V73+**, and 16a4w block quant needs **V69+**. QCS6490's HTP is an older arch, so confirm its version against these before relying on the default weight-sharing decoder path; you may need to disable weight sharing or adjust the quant recipe.

Want me to trace into `MultiModalManager.compile()` (the wrappers file) to show the exact partitioner/to_executor calls per component, or walk through how the three `.pte`s get stitched together in the multimodal runner?

Now lets discuss about prefill in cpu + remaining decoding in npu,
How to acheive this , what code changes we need to do in what files

Analyzing code modifications for CPU-NPU workload distribution >

This is the interesting part — and where the architecture fights you a bit. Let me verify exactly how the current code structures the prefill/decode graphs before I tell you what to change, because the weight-sharing and KV-cache handoff details matter here.

Investigated decoder graph compilation strategy and weight sharing mechanisms >

Two decoder graphs: `kv_forward` (decode) and `prefill_forward`. Right now `compile()` hands both the same QNN spec. Let me see how `MultiModalManager` consumes those specs and handles weight sharing — that's the crux.

Located MultiModalManager file and identified necessary modification points >

`MultiModalManager` is in `wrappers/llm_wrappers.py`. Let me read its structure and the decoder compile path.

Identifying critical function hardwiring graphs together into QNN >

The `HybridTextDecoder.compile()` (line 903) is the heart of it. Let me read it.

Analyzed architectural constraints and identified necessary code modifications >

This tells me what I need. Let me confirm the lowering util is QNN-only and check whether any CPU-fallback mechanism already exists in this harness.

Architected hybrid CPU-NPU split strategy with file-level implementation details >

Good — I traced the actual lowering path, and the honest headline is: **this is not a flag, it's surgery**. The harness is built to compile prefill *and* decode into a single QNN-delegated multi-method `.pte` with weight sharing between them. Let me show you exactly where that coupling lives and what you'd have to break.

Why the current design fights you

In `HybridTextDecoder.compile()` (`wrappers/llm_wrappers.py`, ~line 998), both graphs are lowered in **one call**:

```
python

edge_prog_mgr = to_edge_transform_and_lower_to_qnn(
    module=dict(zip(graph_names, [model.decoder for model in models])), # kv_forward
    inputs=dict(zip(graph_names, example_inputs)),
    compiler_specs=dict(zip(graph_names, data.compile_spec)),
    ...
)
```

`to_edge_transform_and_lower_to_qnn` comes from `backends/qualcomm/_passes/qnn_pass_manager` — it **only** lowers to QNN. There's no backend selection per graph; feeding it `prefill_forward` means prefill goes to HTP. Three things are entangled here:

1. **Single lowering call** produces one `.pte` with both methods, both QNN-delegated.
2. **Weight sharing** (`use_weight_sharing=True` in `llama.py`'s `compile()`) requires both graphs on the same QNN context. CPU prefill breaks that — you'd carry the decoder weights twice (fp on CPU + quantized on NPU).

3. KV-cache handoff is the real killer (more below).

One thing worth knowing: the harness's notion of "CPU" is **per-op fallback**, not whole-graph. The `skip_node_op_set={"llama.fallback.default"}` arg and the `exir_ops.edge.llama.fallback.default` tagging (used today for the embedding gather) let specific ops stay on the CPU portable backend *inside* an otherwise-QNN graph. There is no built-in "run the whole prefill graph on CPU" mode.

The file-by-file changes for a true split

1. `llama.py` — `compile()` (~lines 161-184)

Stop giving both graphs a QNN spec, and kill weight sharing:

```
python

elif modality == TEXT_DECODER:
    backend_options = generate_htp_compiler_spec(
        use_fp16=args.use_fp16,
        use_multi_contexts=decoder_model_config.num_sharding > 1,
        use_weight_sharing=False, # <-- can't share across CPU/NPU
    )
    # Build a QNN spec ONLY for the decode graph (kv_forward).
    # Leave prefill_forward without a QNN spec (None) so it can be
    # routed to a CPU backend in HybridTextDecoder.compile().
```

Note `DECODER_GRAPH_NAMES = ["kv_forward", "prefill_forward"]`, so index 0 is decode, index 1 is prefill — you key off that.

2. `wrappers/llm_wrappers.py` — `HybridTextDecoder.compile()` (~line 998)

This is the main surgery. Split the single lowering into two:

```
python

# decode (kv_forward) -> QNN/NPU, unchanged
decode_edge = to_edge_transform_and_lower_to_qnn(
    module={"kv_forward": self.decode.decoder},
    inputs={"kv_forward": self.decode.export_input},
    compiler_specs={"kv_forward": data.compile_spec[0]},
    constant_methods={**self.decode.meta},
    ...
)
```

```
# prefill (prefill_forward) -> CPU
from executorch.exir import to_edge
from executorch.backends.xnnpack.partition.xnnpack_partitioner import XnnpackPartitioner
prefill_edge = to_edge(torch.export.export(self.prefill.decoder, self.prefill.export_args))
prefill_edge = prefill_edge.to_backend(XnnpackPartitioner()) # or leave undelegated
```

Then either emit two `.pte` files or merge methods into one. The

`ExecutorchBackendConfig` currently uses `alloc_graph_input=False, alloc_graph_output=False` + `BuildQuantIo()` because QNN uses RPC shared buffers — your CPU prefill graph needs **normal** memory planning, so it can't share that config.

3. The runner (`qnn_multimodal_runner.cpp` + the `extension/llm` runner)

The runner calls the prefill method, then loops the decode method. If prefill is XNNPACK/portable and decode is QNN, the binary must **link and register both backends** (today it only pulls in QNN). And the KV buffers passed from prefill→decode currently live in QNN RPC shared memory; CPU output won't, so the I/O plumbing between the two calls changes.

The part that actually decides success: the KV-cache boundary

This is where your past 8a8w/16a16w garbage-output experience comes straight back. The QNN decode graph expects KV-cache **inputs** as quantized tensors with specific scale/zero-point baked in from `SmolVLMQuantRecipe`. CPU prefill emits fp (or differently-quantized) KV. If scales don't match, decode produces garbage — no error, just nonsense.

The current code has `_encoding_override()` (~line 813) precisely to force prefill and decode to share quant encodings *because they're both on QNN*. Split the backends and **you lose that automatic alignment** — you must manually quantize CPU prefill's KV output to match the decode graph's input quant params, or insert an explicit quant op at the handoff.

My honest recommendation

For a 500M model on QCS6490, I'd push back on the premise before doing this surgery. The hybrid AR-N prefill is already a separate graph optimized for the NPU; CPU prefill mostly helps when NPU prefill **OOMs the 4GB HTP context** or hits unsupported dynamic shapes. If it's memory, bumping `num_sharding` or keeping `prefill_ar_len` small (it's already 16 for SmolVLM) is far cheaper than a backend split. If it's a specific op choking

the HTP during prefill, the **op-fallback path** (tag those ops to `llama.fallback.default`) gets you most of the benefit with none of the KV-handoff risk.

If you genuinely need whole-graph CPU prefill, the two-separate-`.pte` route in changes 1-3 is the cleanest, with the KV quant alignment in step 4 as the make-or-break.

What's driving the CPU-prefill requirement — are you hitting an HTP memory/context error, slow prefill latency, or an unsupported-op failure during prefill lowering? That determines whether you want the full split or just selective fallback.

The reason for cpu prefill is bad performance with whole decode on npu even with 16a8w , I want prefill on cpu with fp32

Analyzing KV cache quantization behavior in decode operations >

Let me make sure I get the key technical claim right before advising you, because it changes whether your plan actually helps. Let me check how the KV cache is quantized at the graph boundary and what `--use_fp16` does.

Diagnosing prefill quantization issues and crafting solutions >

Now I have what I need. Let me confirm `--use_fp16` is an actual user flag and find the diagnostic eval flag wiring.

Verified KV quantization persists; diagnosed alternatives to CPU prefill approach >

Before you build this, one correction that reframes the plan — and it's verified in the code, not from memory.

fp32 prefill does *not* escape KV-cache quantization

I checked `_tag_ios` and `_save_output_kv_cache_quant_attrs` in `llm_wrappers.py`. The KV cache is a **quantized I/O**: both the prefill graph's KV *output* and the decode graph's KV *input* are tagged `fixed_point_type["kv_type"]` (16-bit under 16a8w). The cache buffer is stored quantized.

So if you run fp32 prefill on CPU, its fp32 KV output **must be re-quantized to** `kv_type` **before the NPU decode graph can read it**. You get fp32 *compute* during prefill (cleaner

KV values before that final quantization, cleaner first-token logits), but the cache decode actually reads is still 16-bit.

That matters because of where 16a8w usually loses quality: 16-bit KV is nearly lossless, so the corruption is rarely in KV storage. It's far more often the **8-bit weights** in the decode-time matmuls — and fp32 prefill does nothing for those. Decode is still 16a8w. So there's a real chance your plan adds a lot of engineering and barely moves quality.

Diagnose before the surgery — the harness already has the tool

Run the built-in 3-phase eval, which the README documents and I confirmed is wired through `--eval_methods`:

```
bash

python ../llama.py ... --decoder_model smolvlm_500m_instruct \
  --model_mode kv --eval_methods tasks_eval sqnr_eval --tasks wikitext --limit 1
```

This scores **prepare_pt2e (CPU FP) → convert_pt2e (CPU QDQ) → QNN on device**. Read the gaps:

- Big drop at **FP → QDQ**: it's quantization. fp32 prefill won't fix decode; you need fp16 or mixed precision.
- Drop only at **QDQ → QNN**: it's HTP execution/layout, not quantization at all — neither fp32 prefill nor a better recipe helps; it's a backend issue.

If "bad performance" means **latency** rather than quality, stop — CPU fp32 prefill of a 64-image-token + text sequence will be *slower* than the NPU, so the plan is backwards. I'm assuming you mean output quality; correct me if not.

Two one-flag alternatives that match "I want fp quality"

`--use_fp16 (-F)` — verified: this sets `skip_quantize[TEXT_DECODER]=True` and `use_fp16=True`, running the **entire decoder in fp16 on HTP, no PTQ at all**. This is the actual "I don't want quantization quality loss" button, and it hits the 8-bit-weight problem that fp32 prefill misses. Both prefill and decode in fp16 on the NPU. For 500M on QCS6490, just measure latency/memory — fp16 is slower than 16a8w but may be perfectly fine, and it's one flag versus a CPU/NPU split.

`--quant_recipe_suggestion` — runs the layer-wise SQNR analyzer, emits `{model}_suggest_recipe.py` with the sensitive groups bumped to higher precision.

Targeted fix if a few specific layers are producing the garbage.

If you still want CPU fp32 prefill specifically

Everything from last turn holds, now specialized:

- **Prefill graph:** skip PTQ (`skip_quantize` for the prefill method), lower with `to_edge(...).to_backend(XnnpackPartitioner())` instead of `to_edge_transform_and_lower_to_qnn`. Normal memory planning (drop the `alloc_graph_input=False` shared-buffer config).
- **Decode graph:** 16a8w QNN, unchanged.
- **The boundary (make-or-break):** quantize prefill's fp32 KV output to the decode graph's `kv_type` scale/zero-point. Those params are exactly what `_save_output_kv_cache_quant_attrs` captures (`QCOM_SCALE`, `QCOM_ZERO_POINT`, `QCOM_DTYPE`). If the scale you apply doesn't match what the decode graph was calibrated to expect, you get garbage — same failure mode you already know.
- `use_weight_sharing=False`, decoder weights carried twice (fp32 CPU + quantized NPU).
- Runner links **both** XNNPACK and QNN backends; KV buffers no longer in QNN RPC shared memory.

My honest read: run the 3-phase eval first, then try `--use_fp16` for the decoder. If that recovers quality at acceptable speed, you've solved it with one flag and skipped the entire backend-split and KV-requantization risk. Want me to pull the exact `kv_type` / `io_type` definitions from `SmolVLMQuantRecipe` so you can see what precision the KV cache is actually getting?

